



סמל מוסד: 240457

שם מכללה: אדיר במרום

שם פרויקט: **פרוטוקול שידור אודיו UBASS**

שם סטודנט: עמישב כהן

ת.ז: 215754953

מנחה: דרור בן עמי

תאריך: 08/01/2026

1. תיאור הנושא..... 3
2. רקע תיאורטי בתחום הפרויקט 4
3. תיאור הפרויקט 6
4. הגדרת הבעיה האלגוריתמית..... 8
5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות..... 9
6. הליכים עיקריים בתחום למידת מכונה 14
7. הליכים עיקריים בתחום מערכות הפעלה/רשתות מחשבים/תקשורת נתונים/אבטחת מידע 16
8. תיאור פרוטוקולי תקשורת..... 16
9. פיתוחים עתידיים..... 16
10. תיאור טכנולוגיה הנדסה..... 17
11. מסד נתונים..... 17
- ביבליוגרפיה..... 17
12. לוחות זמנים..... 18
13. מנחי הפרויקט..... 18
14. חתימת הסטודנט..... 18
15. חתימת רכזת המגמה..... 18

1. תיאור הנושא

פרויקט זה עוסק בפרוטוקולי תקשורת לשידור מדיה. פרוטוקול תקשורת הוא, בבסיסו, שפה מוסכמת בין שני מכשירים או יותר. כדי להעביר מידע בין מכשירים, הם צריכים תחילה להסכים על השפה בה ידברו ולאחר מכן יוכלו לתקשר באמצעותה. פרוטוקולי תקשורת הם הבסיס לכל האינטרנט כפי שאנחנו מכירים אותו, וכל מכשיר שרוצה לשלוח מידע או לקבל מידע דרך הרשת צריך להשתמש בפרוטוקול תקשורת כלשהו.

מטרתו של פרוטוקול תקשורת היא לספק את המוסכמות והמבנים שיאפשרו למכשירים לשלוח מידע בצורה יעילה, אמינה ובטוחה. פרוטוקולים שונים עושים זאת בדרכים שונות או שמים דגש על אלמנטים מסוימים יותר מעל אחרים. חלקם מיועדים לתקשורת גנרית וכללית, ולאחרים יש מטרה מסוימת והם מנסים למלא נישא מסוימת בצורה יעילה.

פרוטוקול תקשורת לשידור מדיה הוא פרוטוקול תקשורת המיועד לשליחת מידע רציף לאורך זמן. נישא זו שונה משאר הפרוטוקולים מכיוון שבה לרוב השליחת מידע היא חד צדדית - צד אחד משדר, והצד השני מקבל, לעומת תקשורת יותר סימטרית בה שני הצדדים משתתפים באופן יחסית שווה.

האתגרים המרכזיים שצצים בפיתוח ותחזוקת פרוטוקול הם, בין היתר, לוודא שהוא אמין - כלומר שכל המידע שנשלח מגיע ושהוא תקין. פרוטוקול TCP ידוע בתור פרוטוקול בסיס טוב לאמינות, מכיוון שהוא מוודא שכל החבילות שנשלחו הגיעו ליעדם, אבל התהליך הזה דורש יותר זמן ומורכבות. וכמובן יעילות - הפרוטוקול יכול לבצע את מה שהתבקש ממנו בצורה יעילה ומהירה ככל האפשר, גם ברמת התקשורת וגם ברמת החישוביות. פרוטוקול UDP ידוע בתור פרוטוקול בסיס טוב למהירות, מכיוון שהוא כולל כמות מינימלית של לוגיקה ולא מוודא הגעה, אך דבר זה פוגע באמינות שלו.

כשמדובר בפרוטוקולים לשידור מדיה, האתגרים באים לידי ביטוי באופן שונה. הצורך באמינות משתנה בהתאם למדיה המשודרת - לפעמים ניתן לאבד כמות קטנה של מידע מבלי לפגוע באיכות התקשורת, אך זה לא המצב לכל סוגי המדיה. בנוסף, בגלל כמות המידע הגדולה הנשלחת באופן חד צדדי, פרוטוקולים לשידור לרוב קשים יותר לייעול.

2. רקע תיאורטי בתחום הפרויקט

כל פרוטוקול עוסק בהעברת חבילות. חבילה היא מקבץ קטן של מידע שכולל בתוכו

- Headers: כותרות, מידע על החבילה עצמה, למשל כתובת היעד אליה החבילה צריכה להגיע או כל פרט מידע אחר אותו הפרוטוקול רוצה להעביר.
- Payload: מטען החבילה, המידע עצמו אותו החבילה מעבירה.

כיום, כל פרוטוקולי התקשורת בנויים על גבי פרוטוקול Internet Protocol - IP. פרוטוקול זה מאוד מינימלי ודורש רק את המידע הבסיסי ביותר שנצרך כדי להעביר - כתובת היעד, כתובת המקור ועוד כמה פיסות מידע בסיסיות. עליו נבנו פרוטוקולים שונים רבים, והם הוסיפו כותרות משלהם בנוסף לכותרות של פרוטוקול ה-IP, ואלו נשמרים בתוך המטען של חבילת ה-IP. ניתן להמשיך כך, פרוטוקול HTTP/2, אחד הפרוטוקולים הידועים ביותר כיום, נבנה על בסיס TCP, שנבנה על בסיס IP. כלומר, המטען של IP יכול את הכותרות של TCP ואת מטענו, והמטען של TCP יכול את הכותרות של HTTP/2 ואת מטענו.

היכולת הזאת להוסיף עוד חוקיות ופונקציונליות על גבי פרוטוקול קיים מאפשרת למפתחים ליצור פרוטוקולים ייחודיים ומורכבים ללא הצורך להמציא את הגלגל כל פעם מחדש. כיום קיימים פרוטוקולים רבים ושונים, חלקם ממוקדים בתחומים ספציפיים וחלקם מיועדים לשימושים כלליים. לכל פרוטוקול יש שיטות שונות לתקשורת, אבטחה, בדיקות תקינות ועוד. רוב הפרוטוקולים בנויים על גבי הפרוטוקולים UDP ו-TCP - שני פרוטוקולים מינימליים שמספקים פונקציונליות בסיסית אך שימושית כתשתית לפרוטוקולים מורכבים יותר.

תחת הקטגוריה של פרוטוקולי תקשורת, קיימים פרוטוקולים המיועדים לשידור. כשמדובר בשידור בתחום התקשורת, הכוונה היא לשליחה מתמשכת של מידע המיועד לקריאה בזמן אמת. למשל בטלוויזיה, המידע נשלח מהאולפן בו הוא מוקלט אל עשרות אלפי מכשירים ברחבי הארץ בזמן אמת. זהו שידור. דוגמאות נוספות יהיו בשיחות וידאו או אודיו (לדוגמה zoom), ושירותי סטרימינג כמו Netflix לוידיאו או Spotify לאודיו. בתחום זה צצים אתגרים ייחודיים לו. הצורך בשליחה או קבלה שוטפת של מידע יכול לגרום לעומס, וככל שמסת המידע עולה, ככה יותר קשה לוודא שחבילות לא נאבדות בדרך. הכמות מעלה את הסיכויים שחבילה תלך לאיבוד, אך גם גורמת לבדיקה שוטפת שכל חבילה הגיע להיות יקרה הרבה יותר.

לסיכום, יש כיום מגוון רחב של פרוטוקולים, אך כמעט כולם צמחו מאותם שורשים - IP ל-TCP או UDP. ולכולם המטרה המשותפת - להעביר מידע ממכשיר אחד לאחר. חלקם שמים דגש על יעילות, אחרים על אמינות, ולרוב מחפשים את האיזון הנכון בין השניים. בתוכם יש פרוטוקולים מסוימים המיועדים לשידור מדיה, ועליהם האתגרים הקיימים מקשים אפילו יותר.

כיום קיימים לא מעט פרוטוקולים לשידור מדיה, אך אף אחד מהם לא מתמקד בשידור אודיו. הרבה מהם תומכים בשידור אודיו, אבל הם מגיעים עם זמן שיהוי גבוה או מגבלות אחרות שמקשות על השימוש בהם בסיטואציות מסוימות. כמה דוגמאות לפרוטוקולים מפורסמים בתחום:

שם הפרוטוקול	יתרונות	חסרונות
WebRTC	- זמן שיהוי מאוד נמוך (<1 שניות) - תמיכה טובה בהרבה מכשירים - הצפנה מובנית	- מיועד לסיטואציות בהן P2P עובד טוב, לשידור במודל של שרת-לקוח פחות אופטימלי - מיועל בעיקר לדפדפנים ופחות להתקנה ישירה
HLS	- עובד בכל רשת שבה firewall מאפשר הודעות http, דבר יותר נפוץ מכמעט כל אופציה אחרת	- זמן שיהוי מאוד גבוה (15-30 שניות) - הבנייה על HTTP גורמת לתוספת עבודה שלא בהכרח נחוצה לשידור מדיה
RTMP	- זמן שיהוי נמוך (2-5 שניות)	- לא נתמך יותר על ידי הרבה דפדפנים ומכשירים מאז שFlash Player הפסיק לעבוד - דורש שרת ייעודי
RTP	- מאוד גמיש - נבנה במיוחד למדיה בזמן אמת - זמן שיהוי נמוך עד בינוני (2-10 שניות)	- דורש פרוטוקולים נוספים לפעולה (RTCP, RTSP)

נושאים רלוונטיים נוספים:

1. UDP - User Datagram Protocol: פרוטוקול שמהווה לבסיס להרבה פרוטוקולים אחרים. הוא מאוד מינימלי ונקרא חסר חיבור מכיוון שהוא בנוי לשלוח הודעה אל היעד בלי ליצור קשר מראש ובלי לוודא הגעה. הוא מאוד מהיר אך לא אמין מאותן סיבות.

2. TCP - Transmission Control Protocol: פרוטוקול זה בדרך כלל נכלל יחד עם UDP בתור פרוטוקול בסיסי, והוא כנראה הפרוטוקול ששימש לבסיס לכמות הגדולה ביותר של פרוטוקולים אחרים. הוא מבוסס חיבור, כלומר לפני שמכשיר שמשמש ב-TCP שולח הודעה הוא יוצר חיבור עם המכשיר איתו הוא רוצה לדבר, והוא יכול לשמור על החיבור פתוח ליותר מהודעה אחת. הפרוטוקול כולל וידוא הגעה של הודעות ואסטרטגיות נוספות במטרה לוודא אמינות. לכן הוא גם איטי יותר.

3. תיאור הפרויקט

פרויקט זה מתעסק בפיתוח פרוטוקול יעיל ומינימלי מבוסס UDP למטרת שידור של אודיו (בעיקר שירים) באיכות גבוהה עם זמן שיהוי נמוך מאוד, המיועד לשידור אודיו שונה למספר משתמשים משרת מרכזי, נישה שכיום לא קיימת בשוק. המשתמשים של הפרוטוקול אינם אנשים אלא תוכנות שרוצות לנצל את הפונקציונליות שלו, והוא עובד בתור ספריית קוד עם שכבה חיצונית של API עם מספר פקודות שניתן לבצע. השכבה הזאת חושפת מספר פקודות מצומצם שבו האפליקציה יכולה להשתמש, ומסתירה את כל החלקים הזזים שאותם האפליקציה לא צריכה להכיר. כל הפעולות של שכבת ה-API נשלחות לשכבת המנהל שבה מוחלט בדיוק אילו פעולות לבצע.

לא מעט מהפעולות מניחות שיש לאפליקציה דרך לזהות שירים, בין אם זה בשם, מספר מזהה או כל דרך אחרת. זאת מכיוון שמטרת הפרוטוקול היא לספק שידור אודיו, ולכן הוא מניח שהסכמה על צורת הבקשה כבר התקבלה מחוץ לפרוטוקול הזה.

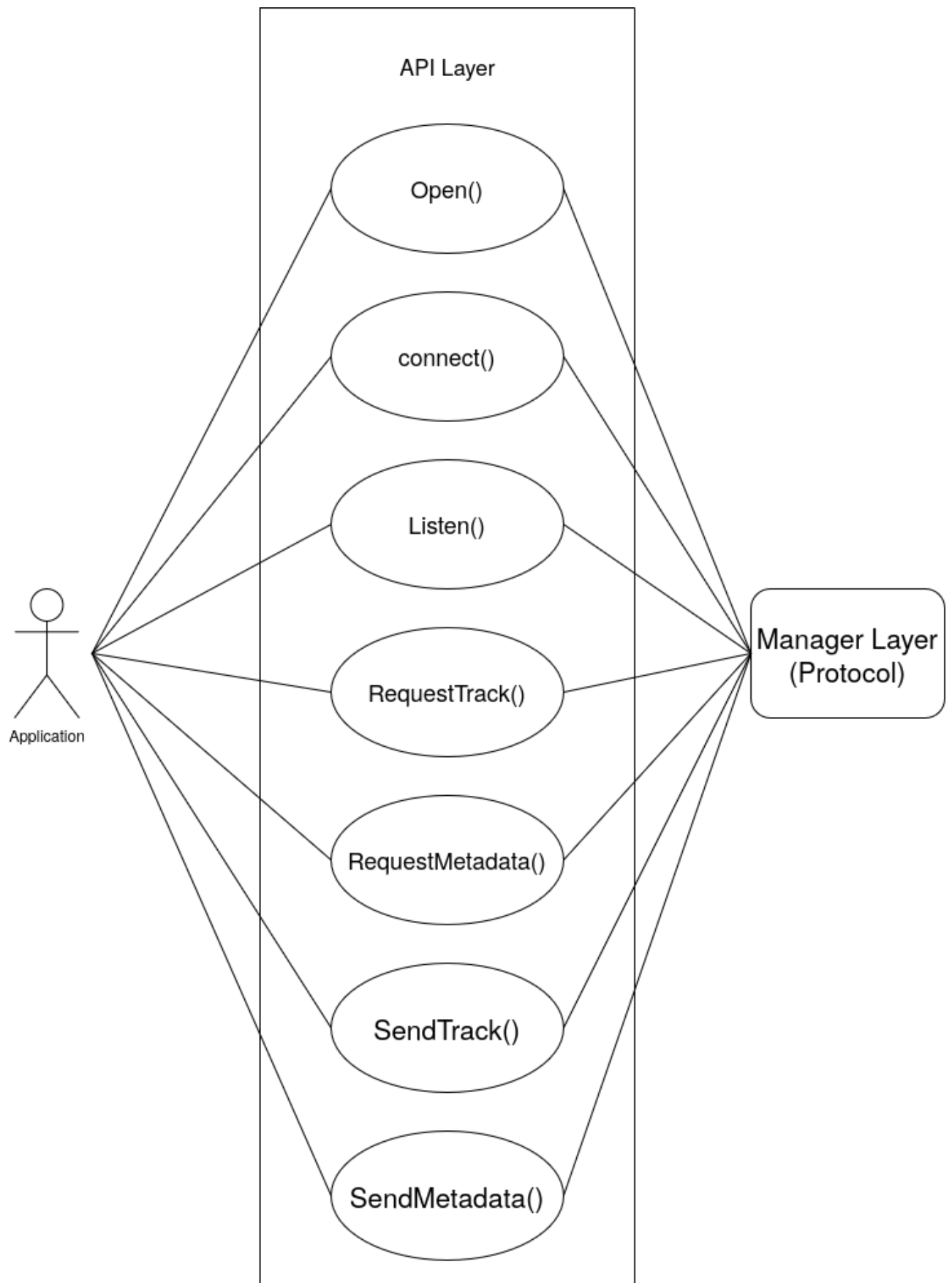
לאפליקציה יהיו מספר פקודות בהן היא יכולה להשתמש, אך תמיד תצטרך להתחיל עם **Open** לפתיחת שלט (controller) שאליו כל שאר הפקודות צריכות להישלח.

לאחר מכן, תוכל לשלוח פקודת **Connect** כדי להתחבר למכשיר אחר עם כתובת לבחירתה, ניתן לדלג על צעד זה ולתת לפרוטוקול לבצע אותו אוטומטית במקרים מסוימים. פקודה זו תחזיר לאפליקציה מספר מזהה של החיבור עם אותו מכשיר.

בנוסף, תהיה לאפליקציה גישה לפקודות הבאות:

1. **Listen**: מאפשר לאפליקציה לחכות עד שתקבל בקשות מכל מכשיר אחר המחובר.
2. **RequestTrack**: שליחת בקשה למכשיר אחר לשידור של שיר מסוים.
3. **RequestMetadata**: שליחת בקשה למכשיר אחר לקבלת המטא-דאטה של שיר מסוים.
4. **SendTrack**: משדר שיר מסוים למכשיר שנבחר.
5. **SendMetadata**: שולח מטא-דאטה של שיר מסוים למכשיר שנבחר.

דיאגרמת UML המתארת את הקשר בין האפליקציה לפרוטוקול דרך שכבת ה-API:



4. הגדרת הבעיה האלגוריתמית

הבעיה האלגוריתמית בבניית הפרוטוקול מורכבת מכמה חלקים:

1. חיבור מהיר בין מכשירים - שיהוי נמוך

- כדי לאפשר לשני מכשירים להתחיל לתקשר בזמן מינימלי, הפרוטוקול ישתמש בכמה אסטרטגיות, ואשלב ביניהן בניסיון להשיג זמן השהיה מינימלי:
- **ORTT (Zero Round Trip Time)**: שילוב הודעת הלקוח הראשונית עם מידע נוסף והצפנה, המאפשר לשרת להתחיל לשלוח נתונים כבר בתגובה הראשונה, בתנאי שהוא כבר מכיר את הלקוח. צריך לוודא שאופן הפיתוח של האסטרטגיה הזאת לא יחשוף את הפרוטוקול לפרצות אבטחה.
- **שילוב ההצפנה בפרוטוקול עצמו**: חיבור handshake ראשוני עם הסכמה על מפתח הצפנה, דבר שמקצר את משא ומתן ההצפנה ומאפשר התחלת שידור מוקדמת יותר. מציאת אלגוריתם הצפנה שמתאים לדרישות האלו ועובד איתן באופן יעיל תהווה חלק קריטי בבניית פרוטוקול אמין ומאובטח.

2. שיפור האמינות של פרוטוקול UDP

- פרוטוקול UDP מהיר אך אינו אמין מטבעו. כדי לשפר את האמינות שלו מבלי לפגוע במהירות, אני אשקול כמה דרכים שונות ואשלב ביניהן כדי להשיג **רמת אמינות גבוהה בלי לוותר על מהירות**:
- **Adaptive Forward Error Correction**: הוספת חבילות שיווי (parity) למידע הנשלח תאפשר ללקוח לשחזר חבילות שנאבדו בלי לשלוח בקשה לשידור מחדש. השרת יתאים את כמות והיחס בין חבילות השיווי לחבילות המידע באופן דינאמי בהתאם למצב החיבור ומספר פרמטרים שונים. הוא ישתמש במודל למידת מכונה כדי לחשב את היחס האופטימלי.
 - מודל למידת המכונה ידרוש dataset ייחודי שכולל מידע ספציפי לדרישות הפרויקט. המודל יאומן על יחס ה-FEC, מדדי איכות חיבור רשת ומהירות שליחה, ולכן נדרש מידע ייחודי ומגוון שיאפשר למודל למצוא מקרי קצה ולפתור אותם באופן נכון ויעיל.
 - **אסטרטגיות שידור מחדש (retransmission) יעילות**: האלגוריתמים יבחרו אילו חבילות יש לשלוח מחדש ומתי, במטרה לשמור על השהיה מינימלית תוך שמירה על שידור רציף וחלק של האודיו.
 - האסטרטגיה צריכה לקחת בחשבון את גודל החלק שנדרש לשלוח מחדש, מספר הבקשות שנשלחו ואופציונלית גם רמת דחיפות אם ניתן לחשב אחת. הצד המבקש יכול לקחת בחשבון את כמות וגודל החלקים שדורשים שליחה מחדש ולשלב מספר בקשות לבקשה אחת אם נצרך.
 - **התאמת האלגוריתמים לתפקיד המכשיר**: לצד הלקוח והשרת יש תפקידים שונים בפרוטוקול זה, לכן ההתאמה של האסטרטגיות האלו לתפקידים תוביל לשיפור משמעותי באמינות והמהירות שבה כל צד יוכל להגיב. לדוגמא לצד הלקוח רמת אמינות עשויה להיות יותר חשובה ממהירות מכיוון שכמות המידע הנשלח ממנו קטנה יותר, אך טעות בבקשה יכולה להוביל לשגיאות גדולות. לכן כדאי להתאים את יחס ה-FEC ופרמטרים אחרים בהתאם לסוג החבילה, מציאת אסטרטגיה יעילה לכך תהווה חלק חשוב מפיתוח הפרוטוקול.
 - **אבטחה**: הפרוטוקול ישתמש באלגוריתם אבטחה מוכן המיועד לשימוש באזורים בהם גם מהירות וגם בטיחות חשובה, כדי לוודא שכל המידע יהיה מאובטח בלי לפגוע במהירות התגובה.

3. אופטימיזציה של הקוד ושימוש בארכיטקטורה א-סינכרונית

ספריית הפרוטוקול תפותח בצורה שתאפשר למספר רב של חיבורים לפעול במקביל באופן יעיל. כדי לוודא זאת, הספרייה תהיה בנויה על בסיס:

- שילוב הרצה א-סינכרונית עם הרצה במקביל כדי לוודא ריצה חלקה ומהירה ככל האפשר
- שימוש באלגוריתם הצפנה שמציב דגש על מהירות ועומד בתנאים הנדרשים לשימוש בטוח ב-ORTT

5. הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

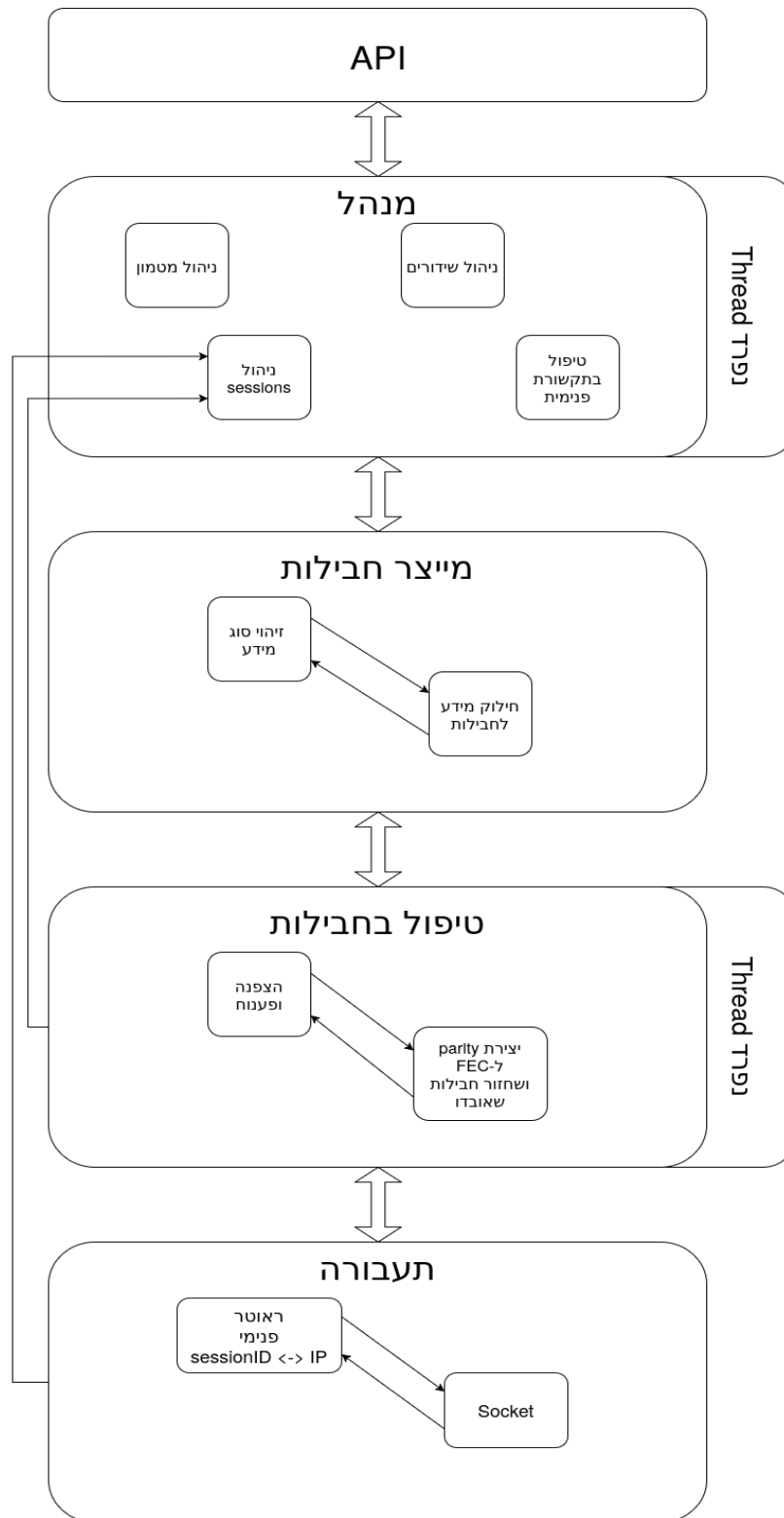
הפרוטוקול מחולק לשכבות, כל שכבה עם תפקידים יחודיים, וכל פעולה תעבור דרך כל שכבה בדרכה לאפליקציה או לשליחה למכשיר אחר, עם שני יוצאי דופן שיוסברו בהמשך. מודל השכבה נבנה כך שבקבלת מידע כל שכבה תקבל את כל המידע מהשכבה שלפניה, תוציא את המידע הרלוונטי לה, תפעל לפיו, ותשלח את השאר לשכבה מעליה. סדר השכבות מוגדר כך שהצד "העליון" יהיה צד האפליקציה והצד "התחתון" יהיה צד התקשורת. בשליחת מידע, כל שכבה תפעל לפי המידע שקיבלה ותוסיף את המידע הרלוונטי אליה. לאחר מכן תשלח את כל המידע אל השכבה מתחתיה.

הסבר על הארכיטקטורה הלוגית:

שכבות הפרוטוקול הן, מלמעלה למטה:

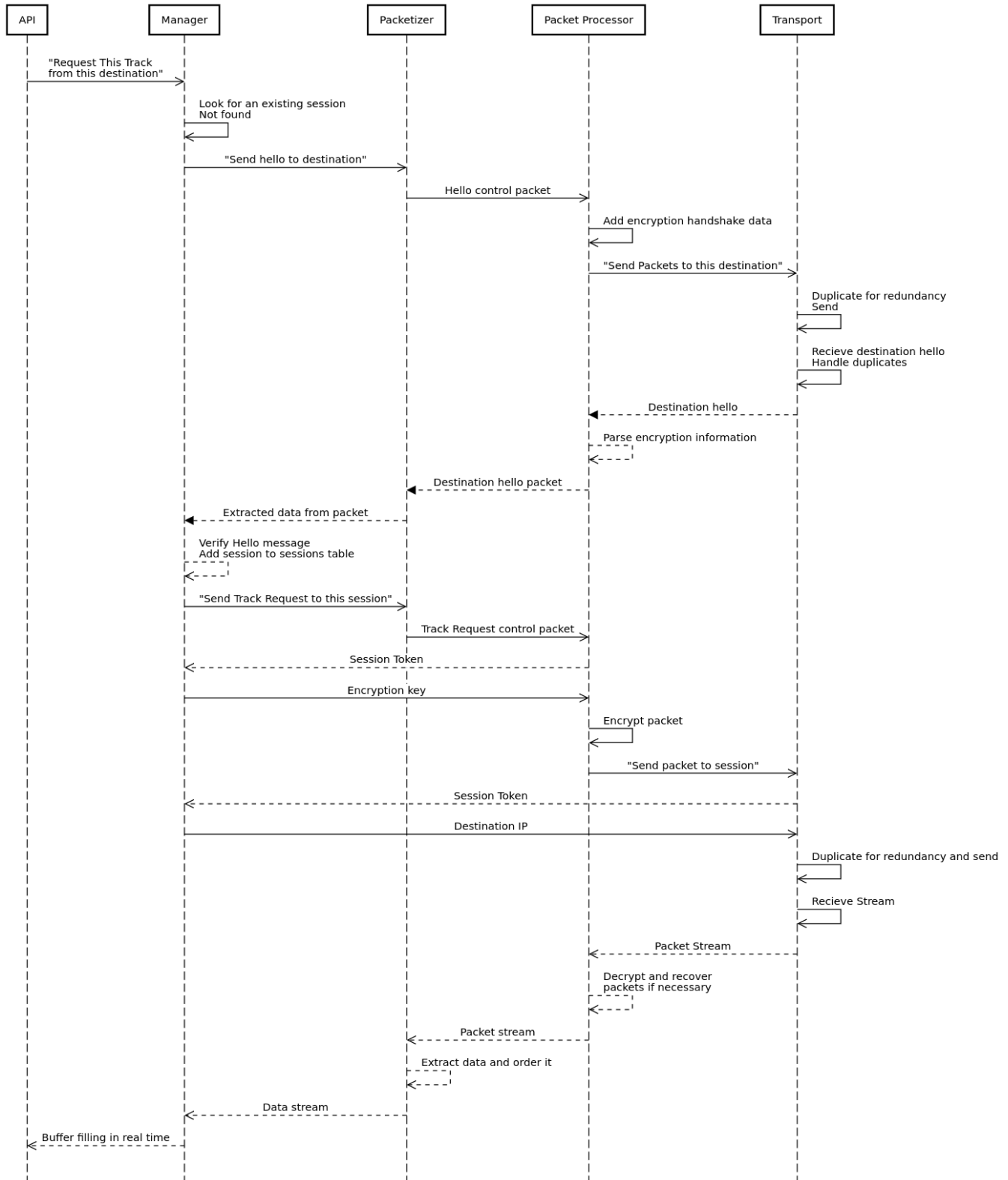
- האפליקציה מדברת עם **שכבת ה-API**, שמתקשרת בין צד המשתמש לצד הפרוטוקול ומסתירה מהמשתמש את רוב המורכבות של הפרוטוקול.
- לאחר מכן יש את **שכבת המנהל**, שכבה זאת אחראית על קבלת החלטות וניהול כל הפרוטוקול - ממעקב אחרי חיבורים עד לטיפול בשגיאות. היא השכבה היחידה שניתן לגשת אליה מכל שכבה, אבל רק במקרים מאוד מסוימים. שכבה זו רצה על תהליכון נפרד, כדי לאפשר לה לעבוד באופן תמידי וחלק בלי להכביד על התהליך הראשי.
- אחריה מגיעה **שכבת ייצור החבילות**. שכבה זו אחראית ליצור ולפרק חבילות לפי הסטנדרט של הפרוטוקול. כאשר תקבל חבילות, היא תוציא מהן את המידע הרלוונטי ותעביר אותו לשכבת המנהל. כאשר תקבל מידע משכבת המנהל, תכניס אותו לחבילה אחת או יותר בהתאם להוראות המנהל ותשלח לשכבה מתחתיה:
- **שכבת עיבוד החבילות** אחראית על הצפנה ו-FEC, כלומר הצפנה ויצירת חבילות parity בקבלת חבילות משכבת ייצור החבילות, ופענוח ושחזור חבילות בקבלתן מהשכבה מתחתיה. שכבה זו יכולה לתקשר ישירות עם שכבת המנהל אך ורק למטרת קבלת מפתח הצפנה. המפתחות נשמרים בשכבת הניהול יחד עם שאר המידע לכל session, לכן שכבה זו צריכה גישה אליהם למטרת הצפנה. הסיבה שתהליכים אלו קיבלו שכבה בפני עצמם היא מכיוון שאלו התהליכים שדורשים הכי הרבה כוח חישובי ולכן הם הופרדו כדי לאפשר להם לרוץ במקביל בקלות. שכבה זו רצה על תהליכון נפרד.
- השכבה האחרונה היא **שכבת התעבורה**, שכבה זו אחראית להעברת וקבלת החבילות ישירות מהרשת. היא אחראית על שליחת החבילות, ובמקרים מסוימים על פעולות פשוטות כמו הכפלת החבילה כדי לוודא הגעה. שכבה זו יכולה לגשת לשכבת המנהל אך ורק למטרת קבלת כתובת ה-IP של החיבור שאליו היא צריכה לשלוח.

ארכיטקטורה לוגית:



תרשים רצף לשליחת בקשה לשידור שיר ממכשיר שעדיין לא נפתח איתו חיבור:

UBASS request Track from a not connected destination



הסבר לתרשים הרצף:

התרשים מראה איך האינטראקציה בין השכבות נראית כאשר האפליקציה מבקשת לבקש שידור שיר ממכשיר אחר שלא מוכר עדיין לפרוטוקול. התרשים מניח שהמכשיר השני זמין ומגיב באופן המצופה ממנו. המנהל מתחיל בלבדוק אם החיבור כבר קיים, כשהוא רואה שהוא לא, הוא שומר את הבקשה בצד ומתחיל בלחיצת יד עם המכשיר השני. לאחר שהלחיצה מסתיימת, המנהל מוסיף את המידע הרלוונטי לרשימת החיבורים.

לאחר לחיצת היד, המנהל שולח את הבקשה לשידור שיר. הפרוטוקול יקבל בחזרה רצף של חבילות מידע עם האודיו של השיר. בשלב הזה שכבת העיבוד חבילה תתחיל לפענח את ההצפנה ולשחזר חבילות בעזרת FEC כשצריך ואפשר, משם שכבת הייצור חבילה תפרק את החבילות ותשלח את האודיו למנהל שיוסיף אותם לחוצץ (buffer) שממנו האפליקציה תקרא את האודיו.

מידע נוסף:

רוב התקשורת מחוץ לשידור תתקיים באמצעות חבילות שליטה (control) שנשלחות כחבילה אחת ולכן במקום להשתמש ב-FEC, שיטה שעובדת רק לכמות גדולה יותר של מידע, שכבת התעבורה משכפלת אותם כדי לוודא שהחבילה תגיע.

הפרוטוקול יכול לקבל ולשלוח גם מטא-דאטה, למידע הזה יש סוג חבילה משלו, והוא יכול להגיע ביותר מחבילה אחת במקרים בהם המטא-דאטה שוקל יותר מהגודל המקסימלי של חבילה אחת. במקרה הזה שכבת הייצור חבילות תחכה עד שכל החלקים יגיעו ותחבר אותם לפני שתשלח שהלאה.

במצב בו המנהל מוצא כבר חיבור עם אותו מכשיר, הוא יפעל בצורה שונה בהתאם לאם החיבור פעיל או לא פעיל, אם החיבור לא פעיל המנהל יחליט לנסות להשתמש באסטרטגיית 0RTT. אם החיבור פעיל ניתן לשלוח את הבקשה ישירות.

מידע תיאורטי רלוונטי להליכים בטכנולוגיות הנדסה מתקדמות

1. FEC - Forward Error Correction

FEC היא טכניקה להוספת חבילות עודף או שיווי (parity) שמיועדות לעזור לשחזור מידע שאבד.

העיקרון:

- שולחים N חבילות + K חבילות שיווי
- ניתן לשחזר עד K חבילות בלי לשלוח בקשה לשידור מחדש

דוגמא פשוטה (XOR)

$$A = 1010$$

$$B = 1100$$

$$\text{Parity} = A \text{ XOR } B = 0110$$

אם A הלכה לאיבוד

$$\text{Parity XOR } B = 1010 = A$$

להפך:

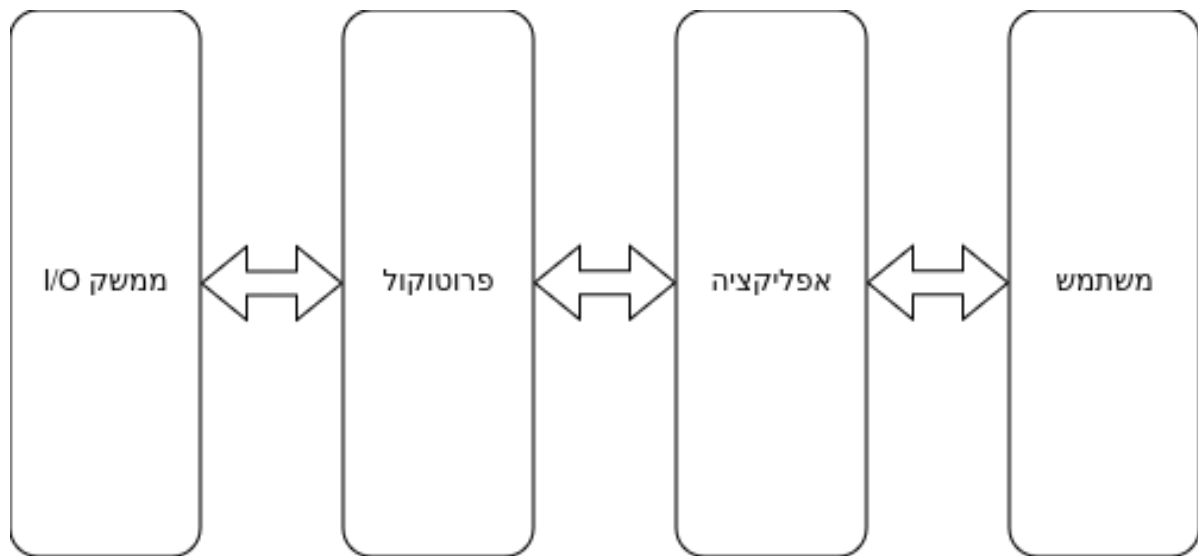
$$\text{Parity XOR } A = 1100 = B$$

ניתן להשתמש גם באלגוריתמים מורכבים יותר כגון ריד-סולומון, BCH, ו-LDPC. שמשתמשים בשיטות יותר יעילות אך יותר מורכבות לביצוע.

2. RTT - Zero Round Trip Time-0

אופטימיזציה זאת מאפשרת לשרת להגיב עם מידע ישר אחרי קבלת בקשה ראשונה אם הלקוח כבר מוכר.
 ללא RTT: לקוח שולח HELLO -> שרת שולח HELLO -> לקוח שולח בקשה -> שרת שולח תגובה
 עם RTT: לקוח שולח HELLO יחד עם בקשה -> שרת שולח תגובה
 ניתן לראות שמידע נשלח ללקוח כבר בסבב הראשון, לעומת האופציה הראשונה שבה התקבל מידע רק אחרי סיבוב וחצי (HELLO + בקשה)
 טכניקה זאת אפשרית רק אם כבר הוסכמה ההצפנה בין שני הצדדים והשרת שמר את המפתח.

ארכיטקטורה פיזית:



הסבר לארכיטקטורה הפיזית:

פרוטוקולים לרוב מאוד אגנוסטיים לארכיטקטורה הפיזית עליה הם רצים כל עוד ניתן למצוא את הכתובת איתה הם רוצים לדבר. מסיבה זאת, הארכיטקטורה היא מאוד פשוטה.
 המשתמש מבצע פעולות עם האפליקציה (למשל בחירת שיר), האפליקציה מבקשת פעולות מהפרוטוקול (בקש את שידור השיר הזה מהמקור הזה), והפרוטוקול בתורו מעבד את הבקשה ומתקשר עם ממשק הקלט/פלט של המחשב כדי לשלוח מידע על גבי האינטרנט. לקבלת מידע (קבלת השידור של השיר) התהליך קורה בכיוון ההפוך: המידע מגיע מממשק הקלט/פלט אל הפרוטוקול שמעבד אותו, משם אל האפליקציה, ומשם אל המשתמש.

6. הליכים עיקריים בתחום למידת מכונה

מטרה:

בפרויקט זה אני אשתמש במודל למידת מכונה כדי לחזות את כמות חבילות המידע לכל חבילת שיווי (יקרא חמ"ש - חבילות מידע לשיווי) הנחוצה כדי להשיג את קצב השליחה הרצוי. מטרת השימוש היא למצוא דרך דינמית אך אמינה להשיג את החיבור הכי אמין שאפשר בלי לוותר על יעילות או מהירות, ובחירת חמ"ש יעיל יכולה להוביל לשיפור משמעותי בתחום.

איסוף מידע:

הנתונים יאספו במיוחד בשביל המודל הזה ממספר מכשירים. ביניהם שרתי ענן, שרתים ביתיים, מחשבים ניידים, וטלפונים שונים. הנתונים שיאספו יכללו נתוני איכות תקשורת, קצב שליחה (בתים לשניה), וחמ"ש, ויאספו על ידי בדיקה של ביצועי הפרוטוקול עצמו בגרסאות הראשונות שלו עם חמ"שים שונים.

ניקוי נתונים:

לאחר מכן, הנתונים ינותחו על מנת למצוא נתונים חסרי השפעה (לכל ערך של הנתון התוצאה זהה כל עוד כל שאר הנתונים זהים), ולמצוא את השילוב הנתונים המתאים ביותר לאימון המודלים שכולל את המספר הרב ביותר של הנתונים שנאספו.

שיטת אימון:

המודל יאומן בשיטת למידה מונחית, שבה יצטרך למצוא דרך להתאים בין המידע שלפיו הוא חוזר לנקודת המידע אותה הוא מנסה לחזות. כדי לבחון את המודל, חלק מנקודות המידע ישמרו בצד והם ישמשו לבחינת המודל לאחר מכן. אם הוא מצליח לחזות את הערכים הנכונים באופן עקבי, הוא יוגדר כמודל מצליח.

בגלל סביבת ההרצה של המודל, ישנן מספר מגבלות על האלגוריתמים האפשריים:

- 1. יעילות גבוהה:** המודל חייב לעמוד בסטנדרט גבוה של יעילות על מנת לוודא שאינו משפיע לרעה על הביצועים של הפרוטוקול כולו. מגבלה זו פוסלת מיד מודלים שלרוב מאוד כבדים כמו רוב רשתות הניורונים.
- 2. צפיית:** המודל צריך להיות צפוי ויחסית קל לפיצוח, על מנת שיהיה אפשר להוכיח באופן גורף שהוא אכן אמין מספיק כדי להיות בשימוש בסביבה בה נכונות היא מאוד חשובה, שכן טעות בחישוב יכולה להוביל לאיבוד חבילות גבוה מהצפוי ובעקבות כך להאטה בקצב השידור. דבר זה פוסל גם הוא רשתות ניוונים, ואף מודלים אחרים כגון SVM.
- 3. שימוש בלמידה מונחית:** מטרת המודל ברורה וצפויה, לכן למידה לא מונחית לא אפקטיבית. בנוסף, למידת חיזוק אינה אופציה הגיונית מכיוון שסימולציה של הפרוטוקול תדרוש הרצה של פעולות קלט פלט, דבר שיאט מאוד את קצב הלמידה מעבר למקובל.
- 4. פלט מסודר ובטוח קבוע:** פלט המודל צריך להיות מוגבל לערכים בטווח ריאליסטי (לא יכולות להיות יותר חבילות שיווי מחבילות מידע, לא יכול להיות ערך לא שלם) ומסודרים לפי גודל (חמ"ש של 20 הוא יותר גדול מחמ"ש של 10), דבר שלא תמיד קיים בכל מודל.

בפרויקט יבדקו שני אלגוריתמים שעונים על כל התנאים הללו, והיעיל ואיכותי יותר מבין השניים יבחר לשילוב לתוך הפרוטוקול:

• ML1 - רגרסיה לינארית:

- רגרסיה לינארית היא מודל ללמידת מכונה הממפה מידע לפונקציה לינארית האופטימלית לייצוגו שתאפשר לחזות מידע חדש על בסיס היחסים בין נקודות המידע השונות במידע המקורי. המודל מניח שיש קורלציה לינארית בין הנתונים, אחרת לא יוכל לבצע חיזויים נכונים.
- **המיפוי נעשה באמצעות פונקציה לינארית** מצורה $y = a_1x_1 + a_2x_2 \dots + b$, כשכל x נקרא פיצ'ר, או נקודת מידע יחודית, וכל a נקרא משקל, והוא נקבע במהלך אימון המודל. y הוא הערך אותו המודל מנסה לחזות, ו- b (נטייה) הוא ערך נוסף שנקבע במהלך אימון המודל.
- **מודל זה עונה על כל הדרישות:** החיזוי שלו מאוד יעיל - סיבוכיות $O(1)$, מכיוון שתהליך החיזוי הוא רק חישוב פונקציה לינארית. הצפיות שלו גם מאוד גבוהה, מכיוון שהחיזוי הוא פונקציה לינארית, וניתן לחשב את החיזוי אפילו עם דף ועט. בנוסף, מודל רגרסיה לינארית בנוי ללמידה מונחית.
- אם ערכי החמ"ש שעליהם המודל יאומן נופלים בטווח הצפוי וערכי התנאי תקשורת יכסו את רוב הטווח הצפוי בתנאים ריאליסטיים, אזי המודל יחזה בכל מקרה ריאליסטי ערך הגיוני, שאותו ניתן לעגל למספר שלם על מנת לקבל ערך חמ"ש סטנדרטי. ולכל מקרה שבו הערך החזוי אינו בטווח, ניתן להניח שמדובר במקרה קיצון ולהשתמש בערכי הקיצון המתאימים (אם החיזוי יותר גבוה מהחסם העליון שהוצב, ערך החמ"ש יהיה שווה לחסם העליון ולהפך בחסם התחתון)

● ML2 - רגרסיה לוגיסטית אורדינלית:

- רגרסיה לוגיסטית אורדינלית היא אלגוריתם סיווג שבו לקטגוריות יש סדר מובנה לתוך ההגדרה שלהם. הצורה שבה זה בא לידי ביטוי היא בשיטת החיזוי והאימון של האלגוריתם.
- **לאימון** תחילה מוצבים לאלגוריתם גבולות (thresholds) כמספר הקטגוריות פחות אחד, כדי לחלק את התוצאות האפשריות למספר הקטגוריות אותן רוצים לבדוק. אחר כך, לכל נקודת מידע אותה יקבל המודל יחשב ניקוד באמצעות קומבינציה לינארית $(score = a_1x_1 + a_2x_2 \dots + b)$, ובו ישתמש כדי לחשב את ההסתברות שנקודת המידע הזאת נכנסת תחת כל קטגוריה באמצעות בדיקת ההפרש בין הניקוד לכל אחת מהגבולות ונרמל באמצעות פונקציית סיגמויד:
- $P(score \leq threshold_i) = \text{sigmoid}(threshold_i - score) - \text{sigmoid}(threshold_i - score)$
- i מייצגת את מספר הגבול, בגבול הראשון לא מפחיתים מהגבול שאמור להיות לפניו.
- לאחר החישוב הזה, האלגוריתם יקח את ההסתברות הכי גבוהה בכל נקודת מידע, ויסכום את ה-NLL שלהם (Negative Log Likelyhood). זה יתן לו ניקוד לכמה מדויק המודל היה בחיזוי שלו, ולפיו יוכל לתקן באמצעות אלגוריתם תיקון כגון gradient descent.
- NLL בבסיסו הוא פשוט חישוב של $-\log(p)$, כאשר p הוא ההסתברות הכי גבוהה לאותה נקודת מידע.
- **לחיזוי** האלגוריתם משתמש באותם שלבים, רק שבמקום לחשב סכום NLL האלגוריתם מחזיר את הקטגוריה שקיבלה את ההסתברות הכי גבוהה לנקודת המידע אותה קיבל.
- **מודל זה גם עונה על כל הדרישות:** הסיבוכיות ריצה שלו היא כמספר הקטגוריות אותן הוא צריך לחזות, ובגלל מגבלות הפרויקט כמות הקטגוריות לא תעלה על 150. בנוסף, הצפיות של המודל אמנם לא חזקה כמו של מודל רגרסיה לינארית, אבל עדיין ניתן לצפות ולהבין את תוצאות המודל, כיוון שהן מבוססות על חישובים

מתמטיים יחסית פשוטים ומעטים. אלגוריתם זה גם הוא משתמש בלמידה מונחית, והוא עונה ישירות על הצורך בטווח מוגבל של ערכי חמ"ש אך בלי לאבד את המשמעות של סדר הערכים.

7. הליכים עיקריים בתחום מערכות הפעלה/רשתות מחשבים/תקשורת נתונים/אבטחת מידע

תקשורת נתונים:

- פרטוקול מותאם אישית
- נבנה על UDP כדי שיהיה בסיס נתמך ומהיר

אבטחת מידע:

- ההצפנה תהיה מובנית ישירות בפרטוקול כדי לוודא שהיא מתאימה לדרישות הפרויקט. היא תיכלל בתהליך ה-handshake של הפרטוקול כדי להמעיט במספר ההודעות הנדרשות לפני שהחיבור נפתח.
- הדרישות לאלגוריתם ההצפנה כוללות מהירות הרצה טובה תוך שמירה על רמת אבטחה גבוהה, יעילות בהצפנה של מידע רציף כמו אודיו תשפיע גם על ההחלטה.
- שלט הפרטוקול יבנה בצורה שבה לגופים חיצוניים תהיה גישה מינימלית למידע שנשמר בו, עם מעט מאוד נקודות גישה לשירות שלכל אחת מהן מטרה מוגדרת ומסוימת.

8. תיאור פרטוקולי תקשורת

פרויקט זה עוסק בפיתוח פרטוקול תקשורת הבנוי על בסיס UDP, לכן התקשורת מנוהלת ישירות על ידי התוכנה עצמה.

9. פיתוחים עתידיים

- הגרסה הנוכחית לא כללה פיצ'רים כגון
 - שימוש ב-adaptive bitrate לשירים
 - אפשרות בחירה לאלגוריתם ההצפנה
 - שימוש בערוצים על מנת לשדר מידע במקביל לאותו חיבור
- ואותם ניתן להוסיף בגרסאות עתידיות

10. תיאור טכנולוגיה הנדסה

- ספריית הפרטוקול נכתבת כולה ב-C לביצועים מקסימליים וניידות
- הספרייה תיעזר במספר ספריות חיצוניות, למשל ספריית ההצפנה שבה הפרטוקול ישתמש וספריות סטנדרטיות כמו stdlib, stdio, ועוד
- צד השרת יפותח בשפה המתאימה לביצועים גבוהים ועם תמיכה ב-async
 - למשל Rust/Go/C++

- צד הלקוח יפותח ב-Java/Kotlin לאינטגרציה קלה עם אנדרואיד, מכיוון שניידים הם פלטפורמה שמשומשת לרוב לשמיעת אודיו בשידור (כדוגמה שירותי סטרימינג כמו spotify, apple music)
- שימוש ב-JNI כדי לגשר בין Java לספריית הפרוטוקול ב-C
- שימוש ב-Track Provider API של PowerAmp
- שימוש בספריה ל-FEC
- שימוש בספריה בפייתון לארגון ואימון המודלים ללמידת מכונה
- לסביבת הפיתוח אשתמש ב-IDE המתאימה לשפה בה אשתמש:
 - Pycharm לפייתון
 - IntelliJ ל-java
 - Android Studio אם מדובר בפיתוח לאנדרואיד
 - Neovim לרוב שאר השפות

11. מסד נתונים

בפרויקט זה אין מסד נתונים.

ביבליוגרפיה

[Error correction code - Wikipedia](#)
[LDAPWiki: 0-RTT Handshakes](#)
[PowerAmp Track Provider API Documentation](#)
[The Top 7 Streaming Protocols for Live Broadcasting](#)
[Linear Regression in Machine learning - GeeksforGeeks](#)
[Ordinal Logistic Regression models and Statistical Software: What You Need to Know](#)

12. לוחות זמנים

	נושא	כמות שעות
1	הצעת פרויקט	30

2	איפיון	40
3	פיתוח ומימוש	
	- פיתוח ספריית הפרוטוקול	300
	- אסיפת מידע לאימון מודל	50
	- אימון מודל ובחינה	85
4	אינטגרציות	120
5	בדיקות ודיבוג	125
6	כתיבת ספר פרויקט	50
-	סה"כ	800

13. מנחי הפרויקט

14. חתימת הסטודנט



15. חתימת רכזת המגמה